

PostgreSQL & Ruby on Rails

The Complete Master Guide: From Architecture to Implementation

1. PostgreSQL Architecture

PostgreSQL uses a **Client-Server model**. A single database cluster is managed by a supervisor process called the "Postmaster".

- **Process-per-connection:** For every client connection, a new backend process is forked. This ensures isolation and stability.
- **Memory Structure:** Utilizes *Shared Buffers* for caching data pages and *WAL Buffers* for transaction logging.
- **MVCC (Multi-Version Concurrency Control):** Allows readers and writers to operate simultaneously without locking. Each transaction sees a "snapshot" of data.
- **Write-Ahead Logging (WAL):** All changes are written to logs before data files, ensuring "Durability" (the D in ACID).

2. Data Types

Category	Types	Details
Numeric	INT, BIGINT, DECIMAL, SERIAL	SERIAL is used for auto-incrementing IDs.
String	VARCHAR(n), TEXT	TEXT is unlimited and performant in Postgres.
Boolean	BOOLEAN	Stores TRUE, FALSE, or NULL.
Temporal	TIMESTAMP, TIMESTAMPTZ, DATE	Always prefer TIMESTAMPTZ for global apps.
Advanced	JSONB, UUID, ARRAY	JSONB is indexed binary JSON.

3. SQL Basics (CRUD)

```
-- CREATE
INSERT INTO users (name, email) VALUES ('John Doe', 'john@example.com') RETURNING id;

-- READ
SELECT * FROM users WHERE active = true ORDER BY created_at DESC;

-- UPDATE
UPDATE users SET last_login = NOW() WHERE id = 1;

-- DELETE
DELETE FROM users WHERE email LIKE '%@spam.com';
```

4. Deep Dive: Joins

Joins allow you to retrieve related data from multiple tables.

A. INNER JOIN

Returns rows only when there is a match in both tables.

```
SELECT users.name, posts.title FROM users INNER JOIN posts ON users.id = posts.user_id;
```

B. LEFT JOIN (Outer Join)

Returns all records from the left table, and matched records from the right. Unmatched right records result in NULL.

```
SELECT users.name, posts.title FROM users LEFT JOIN posts ON users.id = posts.user_id;
```

C. FULL OUTER JOIN

Returns all records when there is a match in either table.

```
SELECT * FROM users FULL OUTER JOIN posts ON users.id = posts.user_id;
```

5. Indexing

Indexes act as a lookup table to speed up data retrieval. Without them, Postgres performs a "Sequential Scan" (reading the entire table).

- **B-Tree:** Default index type for most queries ($=$, $<$, $>$).
- **GIN:** Used for full-text search and JSONB containment queries.

```
CREATE INDEX idx_users_email ON users(email);
```

6. Management Tools: CLI & pgAdmin

| A. CLI (psql)

The terminal-based interface. Fast and scriptable.

```
\l          -- List databases
\c dbname   -- Connect to database
\dt         -- List tables
\d table    -- Describe table structure
```

| B. pgAdmin 4

The graphical user interface. Essential for:

- **Visual Explain:** Seeing a flowchart of query execution.
- **ERD Tool:** Visualizing the database schema.
- **Dashboard:** Monitoring real-time server metrics.

7. Ruby on Rails Implementation

Rails uses Active Record to interface with PostgreSQL.

| The N+1 Problem & Eager Loading

N+1 Issue: Loading a list of objects and then performing a query for each object's association.

```
# GOOD (Eager Loading)
# Fires 2 queries: one for all users, one for all their posts
@users = User.includes(:posts).limit(10)

# SQL Inner Join in Rails
@published_authors = User.joins(:posts).where(posts: { status: 'published' })
```

| PostgreSQL Data Types in Rails Migrations

```
class CreateUserProfiles < ActiveRecord::Migration[7.0]
  def change
    create_table :profiles do |t|
      t.references :user, type: :uuid # Using UUID for foreign keys
      t.jsonb :settings, default: {} # Native JSONB support
      t.timestamps
    end
    add_index :profiles, :settings, using: :gin # GIN index for JSONB
  end
end
```

Final Note: Master the **EXPLAIN ANALYZE** command in both CLI and pgAdmin to ensure your Rails queries are using the indexes you've created.